
RAG 아키텍처와 성능평가
{ Vector • Graph 데이터베이스}

RAG(검색 증강 생성)의 개념과 역할

- RAG (Retrieval-Augmented Generation) 정의
- LLM의 태생적 한계를 보완하기 위해 외부 지식 소스 적극 활용
- 사용자에게 보다 정확하고 최신의 정보를 제공하는 핵심 AI 기술



LLM 단독 사용

- 요리 실력은 뛰어나지만 냉장고에 무엇이 있는지 모르는 요리사와 동일
- 냉장고 확인 없이 요리 진행 → 없는 재료 사용 응답 생성 → 신뢰도 급락



RAG 적용

- 냉장고 문을 열어 실제 재료를 명확히 제시
- 제공된 재료(검색된 지식)로만 요리(응답 생성)하도록 완벽히 통제

LLM 단독 사용의 주요 문제점 (1/2)

① 모델 환각 (Hallucination)



- 확률 기반 텍스트 생성 모델의 근본적 한계 노출
- 사실적 근거 부재 시, 그럴듯하지만 일관성 없는 허위 내용 생성

② 적시성 부족 (Lack of Timeliness)



- 모델 파라미터 증가에 따른 훈련 비용 및 소요 시간 대폭 상승
- 실시간 이슈 및 최신 정보의 모델 반영 지연 현상 발생

③ 데이터 보안 취약 (Data Security Risks)



- 일반 LLM은 기업 내부망 및 민감한 개인 데이터 접근 원천 불가
- 보안 해결을 위한 로컬 저장소 및 로컬 연산 환경 구축 필수

RAG의 보완적 역할



- 외부 지식 베이스의 실시간 연동 지원
- LLM의 태생적 한계(환각, 보안, 최신성) 극복 및 신뢰성 확보

LLM 단독 사용의 주요 문제점 및 RAG의 보완 (2/2)

④ 응답 제한 (Response Limitation)

- RAG 활용 시 검색된 단서 기반으로 LLM의 최종 응답 범위 엄격히 제한
- 원치 않는 방향의 답변 생성 원천 제어 가능

⑤ 신뢰할 수 있는 출처 확보 (Reliable Sources)

- RAG 부재 시 출처가 불분명한 오답 생성 가능성 농후
- 사용자가 지정한 신뢰 출처 정보만 추출하여 답변의 정확성 대폭 향상

⑥ 용어 혼동 방지 (Preventing Terminology Confusion)

- 동일 단어의 맥락별 다의적 사용 시 LLM 자체적 혼동 발생
- RAG를 통해 정확하고 일관된 용어를 보장하는 특정 출처에서 정보 획득

핵심 가치 (Core Value)

- RAG는 LLM의 치명적 한계를 완벽히 보완하는 상호보완적 시스템
- 사용자에게 예측 가능하고 통제된 응답을 제공하는 필수 불가결한 기술

RAG 적용 사례 분석: 최신 정보 질의

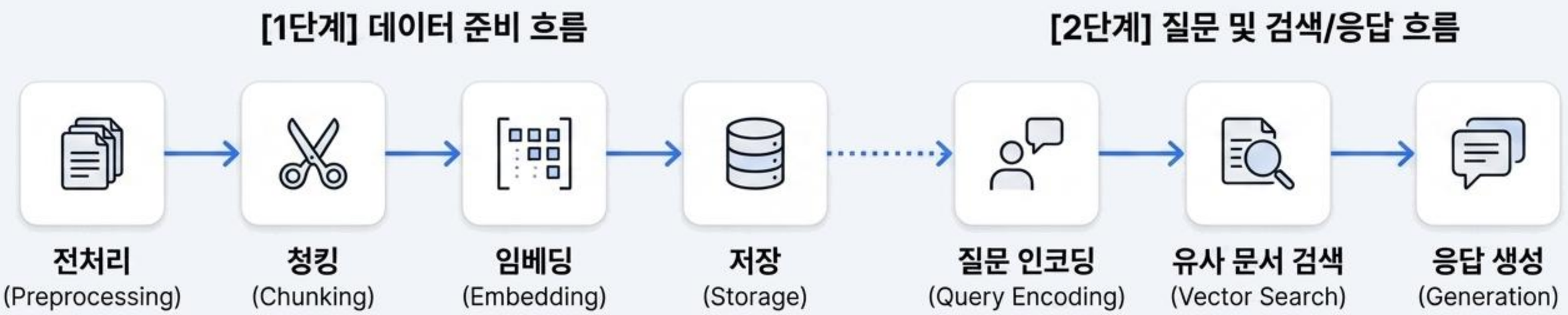
- 전문 지식과 최신 정보가 동시에 요구되는 도메인에서 RAG 효율성 극대화
- 최신 문서(예: 2025년 모의고사)의 실시간 DB화 및 연동 필수
- LLM 단독의 '정보 부재' 상황을 RAG의 '지식 주입'으로 해결

최근 2025년 9월 모의고사 국어 4번 문제를 보여주고 풀이해줘.

해당 모의고사 정보는 알 수 없습니다. (최신 정보 미보유 및 전문적 내용 기술 곤란으로 인한 답변 실패)

문제는 아래와 같으며, 문제 유형과 풀이 과정은 다음과 같습니다. (사전 등록된 전문 지식 기반의 정확하고 상세한 응답 제공)

RAG 아키텍처: 2단계 핵심 구조



핵심 인사이트

- 데이터 준비 단계와 검색/응답 단계는 독립적으로 작동하면서도 유기적으로 결합
- 전체 RAG 시스템의 성능 최적화를 위해 두 단계 모두 반드시 점검해야 하는 주요 대상

RAG 1단계: 데이터 준비 프로세스

① 데이터 수집 및 전처리

- 외부 지식 베이스로 활용할 원본 문서의 수집 및 텍스트 정제
- 불필요한 정보 및 노이즈 제거 (예: HTML 웹 태그 제거)
- 일관된 형식으로 텍스트를 변환하여 데이터 표준화 달성



② 데이터 청킹 (Chunking)

- 수집된 문서를 문장, 문단 또는 특정 길이의 작은 조각 단위로 분할
- 작은 단위 분할을 통해 벡터 검색 단계의 효율성 극대화
- 질문 맥락과 가장 유사한 조각을 정확히 찾아내기 위한 필수 사전 작업



③ 임베딩 및 ④ 데이터 저장

- 각 청크를 컴퓨터가 이해할 수 있는 의미 기반의 벡터값으로 변환 (SentenceTransformers, 임베딩 API 활용)
- 변환된 벡터 데이터를 고속 검색용 벡터 DB에 최종 저장 및 인덱싱 (Weaviate, Pinecone, FAISS 등)



RAG 2단계: 질문 및 검색/응답 프로세스

⑤ 질문 인코딩

- 사용자가 입력한 자연어 질의를 AI 모델이 연산할 수 있는 벡터 형태로 변환
- 인코더(Encoder)가 질문의 내포된 의미와 맥락을 정밀하게 파악
- 검색 대상 주제와 사용자 의도에 완벽히 부합하는 최적화된 벡터 생성



⑥ 유사 문서 검색

- 인코딩된 질문 벡터를 기준으로 벡터 DB 내 유사도 탐색 수행
- 코사인 유사도(Cosine Similarity) 등 고도화된 수학적 알고리즘 활용
- 질문과 맥락적으로 가장 연관성이 높은 핵심 정보 조각(Chunk) 추출



⑦ 응답 생성

- 검색 및 추출된 신뢰성 높은 문서를 컨텍스트로 활용하여 답변 구성
- 프롬프트 엔지니어링 기법 적용을 통해 응답의 최종 형태 및 내용 미세 조정
- 사용자 질의에 구체적이고 맥락적으로 완벽히 부합하는 최종 답변 출력



기존 RAG 시스템의 3가지 주요 한계

① 질의 한계

- 질의 벡터와 문서 조각 벡터 간의 단순 표면적 유사성 매칭에 과도하게 의존
- 사용자가 의도한 복잡한 질의의 미묘한 뉘앙스 차이 및 숨은 의도 이해 실패
- 동일 키워드 포함 시, 문맥적 관련성이 현저히 낮은 정보까지 무분별하게 검색됨



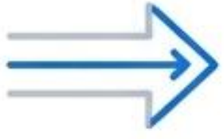
② 검색 한계

- 검색된 다량의 청크 정보를 추가적인 필터링 없이 그대로 LLM의 프롬프트에 제공
- 중복되거나 질의와 관련성이 떨어지는 노이즈 정보가 과다하게 포함됨
- 컨텍스트 윈도우 초과에 따른 연산 자원 낭비 초래 및 잘못된 응답 생성 위험도 증가



③ 선형 프로세스 한계

- '사용자 질문 → 검색 → 응답 생성'이라는 단순하고 경직된 단일 흐름으로만 작동
- 다중 추론이 필요한 복잡하고 다차원적인 사용자 질의 처리에 원천적 부적합
- 다양한 구조를 가진 이기종 데이터 소스의 유연한 통합 및 병렬 처리에 한계 노출



RAG 기술의 발전 방향과 진화

발전 배경

- 생성형 AI 기술 발전에 따라 사용자의 기대치와 요구사항 지속 상승
- 기업용 활용 애플리케이션의 복잡화 및 이기종 데이터 통합 요건 심화

핵심 요약

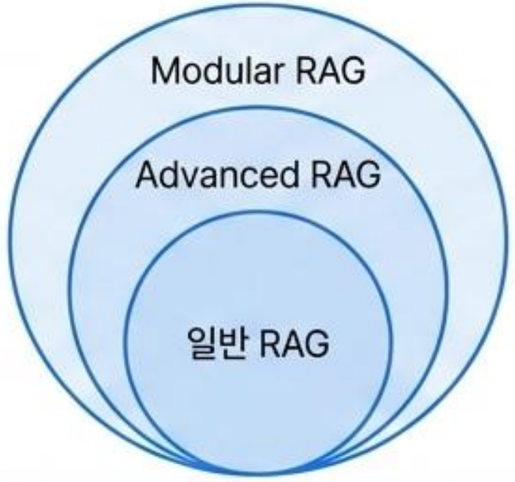
- RAG 기술은 더 이상 단방향 파이프라인에 머물지 않음
- 단순 검색을 넘어 모듈화·패턴화되며, '스스로 추론이 가능한 형태'로 고도화 발전 중

1. **일반 RAG (Naive RAG)**
   가장 기초적인 형태의 검색 후 생성이라는 단방향 선형 흐름

2. **Advanced RAG**
   검색 전후 단계의 데이터 정비 및 검색 전략이 세밀하게 고도화된 구조

3. **Modular RAG**
   다양한 플로우 결합 및 독립적 모듈화를 통한 복합적 추론 아키텍처 완성

RAG 아키텍처 유형별 포함 관계 및 특징 비교



핵심 메시지: 각 RAG 유형은 배타적 선택지가 아닌 진화적 상호 포함 관계를 가짐. 가장 진화된 Modular RAG 내부에는 하위 RAG의 모든 특징이 공존함.

구분	개선의 초점
일반 RAG	검색 후 생성의 기본 흐름 구현
Advanced RAG	검색 전/후 절차 개선 — 데이터 정비, 검색·Search 전략 조정, Collection 분산
Modular RAG	독립 모듈화 + 다양한 FLOW 패턴(분기·조건·반복)
Graph RAG	지식그래프 결합으로 관계 기반 추론 및 지식 확장


RAG 성능 평가
{ Vector • Graph 데이터베이스}

RAG 성능 평가의 두 가지 핵심 축 (검색 vs 응답)

좌측 카드: 검색 평가 (Retrieval)

- **주요 역할: 검색된 문서(청크)의 질의 관련성 및 정답 근거 포함 여부 평가**
  - **핵심 질문 (평가 기준):**
    - 정답 문서가 성공적으로 회수되었는가? (Recall / Hit)
    - 정답 문서가 최상위에 위치하는가? (Precision@K, MRR, NDCG)
  - **실무 적용 포인트:**
    - Top-K 설정 및 청크 크기 결정의 근거
    - 임베딩 모델 및 리랭커(Reranker) 선택의 기준

우측 카드: 응답 평가 (Response)

- **주요 역할: 검색된 컨텍스트 기반 생성 응답의 근거성, 관련성, 정확성 측정**
  - **핵심 질문 (평가 기준):**
    - 컨텍스트에 철저히 근거하여 작성되었는가? (Faithfulness)
    - 사용자 질의 의도에 정확히 부합하는가? (Relevance)
    - 사전 정의된 정답과 일치하는가? (Correctness)
  - **실무 적용 포인트:**
    - 환각(Hallucination) 통제 및 방지
    - 프롬프트 엔지니어링 및 생성 모델 선택의 기준

분리 측정의 필요성: 두 축의 엄격한 분리 평가를 통해서만 실패 원인(검색 누락 vs 생성 오류)의 정확한 타겟팅 및 조치 가능

검색 평가 매트릭스 (Retrieval Metrics) 핵심 유형

컨텍스트 관련성 (Context Relevance / Precision)

평가 목적: 검색된 청크의 사용자 질의 관련성 및 상위 배치 적절성 진단
핵심 지표: Precision@K, Context Precision (RAGAS)
진단 영역: 검색 결과 내 '노이즈(무관 정보)' 발생 수준 파악

컨텍스트 재현율 (Context Recall)

평가 목적: 정답 생성에 필수적인 근거 정보의 검색 결과 내 누락 여부 평가
기준점: 사전 정의된 정답 문서 또는 정답 문장
핵심 지표: Recall@K, Context Recall (RAGAS)
진단 영역: 필요한 정보의 '누락' 발생 수준 파악

적중 여부 (Hit / Hit Rate)

평가 목적: 상위 K개 결과 내 정답 문서의 포함 여부 측정
측정 방식: 쿼리별 이진 판별(True/False) 후 전체 비율 산출
핵심 지표: Hit@K, Hit Rate@K

순위 품질 (Ranking Quality)

평가 목적: 정답 문서의 상단 배치 최적화 정도 평가 (순위별 가중치 차등)
핵심 지표: MRR, NDCG@K, MAP
진단 영역: 검색 결과의 '순서' 및 랭킹 알고리즘 정확도 파악

종합 평가 원칙: '정답 근거 회수율(Recall)'과 '상위 노출 정밀도(Precision·순위)'의 교차 검증 필수

검색 지표(Retrieval Metrics) 정의 및 계산 예시

지표	정의	계산 예시 값
Hit@K	상위 K개에 정답 문서가 1개 이상 존재하면 1, 아니면 0	1 (True)
Precision@K	상위 K개 중 관련 문서 비율	2 / 4 = 0.50
Recall@K	전체 관련 문서 중 상위 K개에 회수된 비율	2 / 3 = 0.67
MRR	첫 번째 관련 문서 순위의 역수(1/rank)를 쿼리 전체로 평균	1 / 1 = 1.00
NDCG@K	관련도를 순위 위치로 할인($\log_2$)해 합산한 DCG를 이상적 순서의 IDCG로 나눔	0.92

예시 조건 적용: K = 4, 전체 관련 문서 3개 존재 / 검색 결과 순위: 1위(관련 ○), 2위(무관 ×), 3위(관련 ○), 4위(무관 ×)

데이터 해석 및 시사점:

- 첫 결과가 정답이므로 MRR은 만점(1.0) 기록
- 관련 문서 1개가 K(4) 범위 밖으로 밀려 Recall(0.67) 수치 저하 발생
- 개선 방향: K값 범위 확대 또는 리랭커(Reranker) 도입 검토 필요

지표 활용 원칙: 단일 지표 의존 불가 (Precision=노이즈, Recall=누락, MRR/NDCG=순서 진단 목적별 복합 사용)

응답 평가 매트릭스 (Response Metrics) 상세 유형

유형	설명	예시 지표
근거성 / 신뢰성 (Faithfulness · Groundedness)	응답의 각 주장이 검색된 컨텍스트로 뒷받침되는지 평가 (환각 탐지)	Faithfulness 점수(0~1), 이진분류(사실/환각)
응답 관련성 (Answer Relevance)	생성된 응답이 질문 의도에 정확히 부합하는지 평가 (동문서답 방지)	Answer Relevancy, 이진분류(관련/무관)
정답 기반 정확성 (Answer Correctness)	사전 정의된 정답(Ground Truth)과 사실적·의미적 일치 여부 평가	EM, F1, Semantic Similarity, Correctness
완전성 (Completeness)	정답에 필요한 정보의 누락 없는 포함 여부 (복수 문서 증합 시 핵심)	Coverage, 이진분류(완전/불완전)
거부 정확성 (Negative Rejection)	컨텍스트 내 근거 부재 시 '알 수 없음'으로의 적절한 답변 전환 여부	Abstention 정확도, 오답 강행률
형식·유창성 (Format · Fluency)	요청 출력 형식(JSON, 표 등) 준수율 및 문장 품질 측정 (보조 지표)	형식 준수율, ROUGE/BLEU/BERTScore
안전성·윤리 (Safety · Ethics)	응답 내용의 차별적, 편향적, 독성 요소 포함 여부 식별	Toxicity, Bias 점수

☆ **우선순위 가이드: 1순위 = Faithfulness (환각 통제) / 2순위 = Correctness·Completeness / 보조 = 유창성 지표**

주요 레퍼런스 프레임워크: RAGAS 핵심 4지표 및 RAG Triad

[RAGAS 4대 핵심 지표]



Context Precision (검색 단계)

관련 청크의 상위 랭킹 배치 적절성 (순위 가중 정밀도)

필요 입력: 질문, 컨텍스트, 정답



Context Recall (검색 단계)

정답 문장의 검색 컨텍스트 내 커버리지 비율

필요 입력: 컨텍스트, 정답



Faithfulness (생성 단계)

응답 주장 중 컨텍스트로 검증 가능한 비율 (환각 측정)

필요 입력: 응답, 컨텍스트 (정답 불필요)



Answer Relevancy (생성 단계)

응답 기반 역생성 질문과 원문 질문 간 임베딩 유사도

필요 입력: 질문, 응답 (정답 불필요)



[TruLens RAG Triad]

구성 요소: Context Relevance (검색) → Groundedness (근거성) → Answer Relevance (응답)

특징: Ground Truth**(정답)** 구축 없이도 근거성 및 관련성 자동 평가 가능

실무 적용: 실시간 운영 환경(Production)의 대규모 모니터링 및 추적에 유리

사례 분석: 성공 기준 및 특수 상황(정답 부재) 대처

[성공 사례 ① (검색 및 응답 동시 성공)]

- **전제 조건:** 좋은 검색은 좋은 응답의 '필요조건' (충분조건 아님)
- **동시 성공 입증 지표 예시:**
  - Hit (True): 상위 K개 내 관련 문서 적중
  - Hit Rate: 평가셋 전체 쿼리 대비 적중 비율 우수
  - Precision@4 (75%): 검색 문서 4개 중 3개가 관련 문서 (노이즈 최소화)
  - Faithfulness (1.0): 모든 응답 주장이 컨텍스트로 증명됨 (환각 0%)
  - Correctness (True): 생성 응답과 사전 정의 정답 완벽 일치

[특수 사례 ② (정답이 존재하지 않는 경우)]

- **상황 진단:** 미래 예측 등 지식베이스에 근거가 없는 질의 (예: 내년 인기 요리?)
- **한계 및 리스크:** LLM의 무리한 추론 시 환각(Hallucination) 발생 빈도 급증
- **평가 기준의 역발상:** 해당 경우 '모른다'고 답변하는 것이 Correctness(True) 획득
- **해결책 및 지표 반영:**
  - **프롬프트 규칙 추가**: 컨텍스트에 없으면 알 수 없다고 명시할 것
  - **검색 유사도 임계값**(Threshold) 엄격 설정
  - **Negative Rejection(거부 정확성) 지표 측정:** 답 없는 질문 10~20% 배정

최적화 사례 ③: 청크(Chunk) 수 축소와 정확도 개선 실험



[1단계: 문제 상황 및 현상 파악]
발생 문제: 검색된 청크 내 관련 정보와 무관 정보가 혼재되어 노이즈 심화
결과 현상: LLM이 무관한 정보에 이끌려 생성 정확도 하락 발생
전제 조건: 관련 문서는 정상 검색됨 (Hit = True, 1위 랭크, MRR = 1.0)

↓



[2단계: 원인 진단 및 가설 수립]
진단 결과: 검색 알고리즘 실패가 아닌 컨텍스트 구성(Top-K)의 문제로 판명
개선 타겟: Precision(정밀도) 개선을 통한 노이즈 제거 시급
현재 상태 (Precision@4 = 25%): 문서 4개 중 3개가 무관 문서 → 응답 품질 저해

↓



[3단계: 최적화 실행 및 결과 모니터링]
조치 내용: 주입 정보 절대량 축소 (Top-K를 4에서 2로 삭감)
개선 결과 (Precision@2 = 50%): 노이즈 절반 감소로 Faithfulness 및 Correctness 상승
주의 사항: K값 축소 시 Recall 저하 가능성 상존 → 교차 확인 필수
핵심 인사이트: LLM에는 핵심 정보만 선별 제공 필요 (리랭커 도입, 임계값 조정 병행)

검색 설정 실험 매트릭스 및 성능-정확도 균형 (Trade-off)

Chunk 크기	청크 수/문서	질문 1	질문 2	질문 3	질문 4	…	정답률
256	447	✓	✓	✓	✓	✓	91%
350	300	✓	✓	✓	✓	✓	94%
420	264	✓	✓	✓	✓	✗	88%
470	250	✗	✗	✓	✓	✗	79%
480	230	✗	✓	✗	✗	✓	72%

100여 개 샘플 질문(Golden Set) 기반 사전 Grid Search 테스트 방법론 적용

[성능(Performance) vs 정확도(Accuracy) 균형점 확보]

* **성능 지표 관점:** 지연시간(Latency), 처리량, 토큰 비용 최적화
(정확도 하락 시 무의미한 빠른 응답에 불과함)

* **정확도 지표 관점:** 검색 정밀도 및 생성 신뢰성 보장
(과도한 무거운 모델/리랭커 적용 시 사용자 이탈 유발)

의사결정 원칙: 두 관점의 비교 표 구축 필수 (예: NDCG +8%p 향상 vs 지연시간 +400ms 증가 동시 고려)

RAG 파이프라인 7가지 실패 포인트 (Failure Points) 맵핑

[검색·데이터 기인 오류 영역]

[생성 모델 기인 오류 영역]

1단계: 사용자 질의
(User Query)

(데이터 및 검색 인덱싱 단계로 진입)

2단계: 인덱싱 및 검색 단계
(Index / Retrieval)

3단계: 생성 단계
(Generation)

4단계: 최종 응답
(Final Response)

FP1. 컨텐츠 없음 (Missing Content): 질문에 대한 해답 자체가 지식베이스에 부재

FP2. 최상위 랭킹 누락 (Missed the Top Ranked): 관련 문서는 있으나 Top-K 밖으로 밀려남

FP3. 컨텍스트 미포함 (Not in Context): 검색은 성공했으나 압축/병합 과정에서 데이터 잘림

FP4. 추출 실패 (Not Extracted): 컨텍스트 내 정답이 있으나 노이즈로 인해 LLM이 추출 실패

FP5. 잘못된 포맷 (Wrong Format): 표, JSON 등 사용자가 지정한 출력 형식 미준수

FP6. 부적절한 구체성 (Incorrect Specificity): 답변 내용이 지나치게 포괄적이거나 불필요하게 상세함

FP7. 불완전한 답변 (Incomplete): 정답 정보 중 일부 내용 누락 (복수 문서 종합 시 발생)

핵심 요건: 발생한 실패가 검색(FP1~3) 기인인지 생성(FP4~7) 기인인지 정확한 단계 특정 선행 필요

실패 포인트(FP) 상세 분석 및 대응 가이드

실패 포인트	내용 및 현상	진단 지표	대응 대안 (해결책)
FP1. 콘텐츠 없음	해답 자체가 지식베이스에 없음 (억지 답변 시 환각)	Hit = False	원천 데이터 확보/정비, '모른다' 응답 유도 프롬프트
FP2. 랭킹 누락	정답 문서 존재하나 Top-K 밖으로 밀림	Recall@K, MRR 낮음	청킹 전략(크기/Overlap) 조정, 임베딩 교체, 리랭커
FP3. 컨텍스트 미포함	검색 후 통합 과정에서 정보 잘림 현상	Context Recall 낮음	리랭킹(Reranking) 도입, 윈도우/압축 전략 조정
FP4. 추출 실패	노이즈/모순 정보로 LLM이 정답 도출 실패	Faithfulness/Correctness 낮음	무관 청크 제거(Precision 향상), 프롬프트 정리
FP5. 잘못된 포맷	특정 형식(JSON 등) 출력 요청 미준수	형식 준수율 저하	출력 형식 명시(프름프트), 구조화된 출력 강제
FP6. 부적절한 구체성	답변이 너무 포괄적이거나 과도하게 상세함	Answer Relevance 저하	질의 재작성(Query Rewrite), Few-shot 예시 제공
FP7. 불완전한 답변	정답은 맞으나 일부 정보 누락 (복합 질의)	Completeness (Coverage) 저하	복잡한 질의를 하위 질문으로 분해(Decomposition)

대응 원칙: 초기 단계의 데이터 준비 및 인덱스 설계(FP1~3)가 견고해야 프롬프트/컨텍스트 최적화(FP4~7) 효과 발휘

AI 검색·응답 평가 진단 Flowchart 및 트러블슈팅



[Phase 1: 응답 평가 수행 (Response Evaluation)]

진행: 최종 응답의 Correctness 및 Faithfulness 기준치 달성 여부 최우선 점검
분기: 지표 양호 시 추가 조치 없이 모니터링 지속 / 문제 발생 시 환각 vs 정답 불일치 원인 분리 후 2단계 진입



[Phase 2: 검색 지표 검토 (Retrieval Diagnostics)]

진행: Hit, Precision@K, MRR, Context Recall 지표 분석
지표 양호 시 (생성 문제 진단): 프롬프트 엔지니어링, 컨텍스트 구성(Top-K, 순서) 수정, 필요시 모델 파인튜닝
지표 불량 시 (검색 문제 진단): 누락(Recall) 문제인지 노이즈(Precision) 문제인지 파악 후 3단계 진입



[Phase 3: 문서 적합성 검증 및 최적화]

문서 적합 시 (기술 고도화): 청킹 기법 변경, 임베딩 모델 교체, 하이브리드 검색 및 리랭커 실험 적용
문서 부적합 시 (데이터 보강): 지식베이스 내 관련 원천 데이터 존재 유무 파악, 누락 인덱스 추가 보충
검증 원칙: 모든 조치는 사전 정의된 Golden Set으로 전후 지표를 재측정하여 회귀(Regression) 방지

평가 데이터셋(Golden Set) 구축 및 LLM-as-Judge 운영



[평가 데이터셋(Golden Set) 구축 가이드]

- **필수 구성 요소:** 질문 + 정답(Ground Truth) + 정답 근거 문서(청크) ID 맵핑 필수
- **구축 규모:** 초기 100~300문항 구축 후 실제 사용자 로그 기반 지속 보강
- **질문 유형 배분 전략:**
  - 단일 문서 사실형 / 복수 문서 종합형
  - 정답 없는 질문(10~20% 할당) / 특정 형식 지정형
- **효율화 방안:** RAGAS TestsetGenerator 등을 활용한 합성 데이터 자동 생성 후 인력 검수
- **버전 관리:** 실험 간 객관성 유지를 위해 데이터셋 버전 고정 필수



[LLM-as-Judge(자동화 평가) 활용 시 주의점]

- **핵심 장점:** 정답(Ground Truth) 없이도 근거성/관련성의 대량 스케일 평가 가능 (운영 모니터링 최적)
- **내재적 편향 리스크:** 자기 선호(Self-preference), 장황함 선호, 위치 편향 발생 가능성
- **대응 및 검증 전략:**
  - 판정 모델과 생성 모델을 물리적으로 분리 적용
  - 명확한 채점 기준(Rubric) 명시 및 이진/소수 등급 단순화
  - 전문가 인력 평가 결과와의 상관계수(Kappa 등) 사전 확인 후 신뢰 부여

주요 RAG 평가 프레임워크 비교 및 지표 종합 레퍼런스

[주요 RAG 평가 프레임워크 비교]

프레임워크	핵심 지표 특징	정답(Ground Truth) 요구도	실무 적합 용도
RAGAS	Context / Answer 정밀도 및 재현율 종합	일부 지표만 필요	가장 범용적 사용, 합성 테스트셋 생성
TruLens	RAG Triad (Relevance → Groundedness)	불필요	정답 없는 실시간 운영 환경 모니터링
ARES	경량 판정 모델 학습 + 예측 추론 (PPI)	소량의 라벨 데이터	LLM API 판정 비용 절감 목적
DeepEval	위 지표 + 환각, 독성, 맞춤 기준(G-Eval)	지표별 상이	단위 테스트 및 CI/CD 파이프라인 통합

권장 파이프라인: 개발 단계(RAGAS/DeepEval 회귀 테스트) → 운영 단계(TruLens 모니터링) 조합

[종합 레퍼런스: 핵심 평가 지표의 실무적 해석]

- **Hit@K [True]:** 검색 최상단 결과에 정답 근거 문서를 성공적으로 낚아 올림
- **NDCG@K [70%]:** 정답 문서가 상단에 깔끔하게 정렬된 이상적 상태 대비 70% 수준
- **Precision@4 [75%]:** 상위 4개 중 3개가 엑기스 정보 (노이즈 25% 혼입 상태)
- **Faithfulness [0.9]:** LLM의 입에서 나온 10문장 중 1문장은 환각(거짓)으로 의심됨
- **Recall@4 [67%]:** 세상에 존재하는 3개의 정답 문서 중 2개만 건짐 (1개 누락)
- **Correctness [True]:** 최종 결과물이 비즈니스 로직 및 사전 정의 정답과 완벽히 일치함


RAG 구축·평가 실습
{ Vector • Graph 데이터베이스}

실습 개요 — 명지대학교 규정 검색 RAG

실습 목표

- 명지대 규정집 PDF 239개를 지식 베이스로 규정 검색 RAG 구축 (LangChain)
- 검색·응답 두 축으로 성능을 점수화 (Golden Set 25문항)
- 평가 결과로 실패 원인을 진단하고 청크 크기·Top-K 결정
- **실습 파일:** 3. RAG_구축과_성능평가.ipynb (Colab, T4 GPU)

사용 도구

- **문서 로딩:** PyPDFLoader
- **청킹:** RecursiveCharacterTextSplitter
- **임베딩:** dragonkue/BGE-m3-ko (1024차원)
- **벡터 DB:** Milvus Lite (COSINE · FLAT)
- **LLM:** 생성 solar-mini4 / 판정 solar-pro4
- **연결:** LCEL 체인 (| 파이프)

Part A. 만들기

수집·정제 ➡ 청킹 4,150개 ➡ 임베딩·저장 ➡ 검색 K=4 ➡ 응답 생성

Part B. 채점하기

Golden Set 25문항 ➡ 검색 지표 6종 ➡ 응답 지표 3종 ➡ 사례·Grid Search ➡ FP 진단·리포트

LLM 단독 vs RAG — 같은 질문, 다른 답

질문: "명지대학교 인공지능·소프트웨어융합대학 학생은 졸업하려면 몇 학점을 취득해야 하나요?"

LLM 단독 (solar-mini4) — 답변 요약

"…전체 이수 학점을 합산하여 **130학점** 이상을 취득해야 합니다. 다만 입학 연도·전공에 따라 달라질 수 있으니 학과에 확인하세요."

✗ **환각:** 실제 규정은 134학점 (학칙 제50조, 2025.4.1. 개정)

RAG 적용 (검색 K=4 + 프롬프트 규칙)

"졸업하려면 **134학점**을 취득해야 합니다.
(출처: 명지대학교 학칙 제50조 제1항)"

✓ **정답 + 근거 출처 제시**

검색된 청크 Top-4 (코사인 유사도)

순위	유사도	청크
1	0.631	학칙 제74조 (입학정원 별표)
2	**0.622**	**학칙 제50조 (학사과정의 졸업) ← 정답 근거**
3	0.597	학칙 제74조
4	0.577	학칙 제74조

프롬프트 규칙 3가지

① [컨텍스트]에 있는 내용만 근거로 답변

② 근거가 없으면 '규정에서 찾을 수 없습니다'

③ 마지막 줄에 (출처: 규정명 제N조) 표기

1단계 데이터 준비 — 수집·정제·규정 맞춤 청킹

① 수집·전처리

- **PyPDFLoader:** PDF 239개 → 규정 1개 = Document 1개 (총 173.8만 자)
- **메타데이터:** 폴더·파일명에서 편 / 장 / 규정번호 / 규정명 추출 (예: 2-1-1 명지대학교 학칙)
- **정제:** 페이지 번호(- 5 -) 제거, 공백 정리
- **조 경계 표시:** '제N조(' 앞에 빈 줄 삽입 → 청킹 기준

② 규정 맞춤 최적 청킹 (500자 / 겹침 50)

1 **개정 이력 주석 제거** (변경·신설·삭제 + 날짜)

2 **조 경계 우선 분할** 빈 줄 → 문장 → 공백

3 **조 머리말 반복** 긴 조의 뒤 조각에도 '제N조(...) ...'

4 **[규정명] 접두어 + article 메타** 출처 추적

→ **청크 4,150개** (chunk_id 예: 500-508)

주석 제거 예시 (학칙 제50조)

전: ...본인의 의사에 따라 졸업을 유예할 수 있다.(변경 2005.3.4, 2006.4.17, 2009.3.1, 2011.5.1, 2012.3.1., 2016.3.1., 2018.7.1., 2019.1.1., 2025.4.1.) ②(삭제 2009.3.1) ③전체 성적이 평균평점 4.0 이상...

후: ...본인의 의사에 따라 졸업을 유예할 수 있다. ② ③전체 성적이 평균평점 4.0 이상...

→ 질문과 무관한 날짜 나열(노이즈)을 지워 임베딩 품질 향상

2단계 검색·응답 — Milvus 저장과 LCEL 체인

```
# ③ 임베딩 + ④ 저장 (한 줄로 컬렉션 생성·저장)
store = Milvus.from_documents(
    chunks, embeddings, collection_name="mju_rules_500",
    connection_args={"uri": DB_URI},
    index_params={"index_type": "FLAT", "metric_type": "COSINE"},
    ids=[c.metadata["chunk_id"] for c in chunks])

# ⑤ 질문 인코딩 + ⑥ 검색: 비슷한 청크 4개
retriever = store.as_retriever(search_kwargs={"k": 4})

# ⑦ 응답 생성: | 로 부품을 차례대로 연결 (LCEL)
rag_chain = ({"context": retriever | format_docs,
              "question": RunnablePassthrough()}
             | RAG_PROMPT | llm | StrOutputParser())

rag_chain.invoke("…졸업하려면 몇 학점을 취득해야 하나요?")
```

LCEL 체인의 흐름

retriever
질문 → 비슷한 청크 4개

↓

format_docs
청크를 <1> <2> … 글로 묶기

↓

RAG_PROMPT
규칙 + 컨텍스트 + 질문

↓

llm (solar-mini4)
답변 생성

↓

StrOutputParser
답변 글자만 꺼내기

평가용 rag() 함수: 답변 + 검색 청크(docs)·검색 시간·전체 시간을 함께 돌려줌 → 채점에 사용

기본(Naive) RAG의 3가지 한계 — 실험으로 확인

① 질의 한계

- 질문: '휴학은 최대 몇 학기까지?'
- Top-5에 학칙 · 학사과정 시행규칙 · 미래교육원 · 특수대학원 규정이 섞임 (4종)
- 질문 의도(학부? 대학원?)가 모호하면 표면적 유사도만으로 엉뚱한 규정 검색

② 검색 한계

질문: '학생 징계의 종류는?' — K를 늘리면?

K	규정 종류	컨텍스트
2	2개	768자
5	2개	2,069자
10	4개	3,905자

- 걸러내지 않고 모두 전달 → K가 커질수록 다른 규정·노이즈·글자 수 증가

③ 선형 프로세스 한계

- 질문: '매 학기 최대 학점만 들으면 졸업까지 최소 몇 학기?'
- 졸업학점 + 학기당 최대 학점을 함께 찾아 계산해야 함
- 검색 → 생성 1회 → 엉뚱한 근거(학·석사연계과정 내규)로 '7개 학기' 답변

→ Advanced RAG(검색 전·후 처리)와 Modular RAG(분기·반복 FLOW)에서 개선

평가 데이터셋(Golden Set) — 25문항 채점 문제지

유형	문항 수	예시 질문
사실형	20	학사과정의 수업연한은 몇 년인가요?
종합형(근거 2)	2	한 번에 몇 학기까지, 통산 몇 학기까지 휴학?
정답없음	3	이번 주 학생식당 점심 메뉴는?

문항 구조 (q12)

```
{"id": "q12", "type": "사실형",
 "question": "학부생은 도서관에서 책을 몇 권,
              며칠 동안 빌릴 수 있나요?",
 "ground_truth": "20책, 14일이다.",
 "evidence": [("5-1-1", "학부과정 학생 : 20책 14일")]}
```

정답 청크 판정 방법

1 evidence = (규정번호, 원문 근거 문구)

2 검색된 청크의 규정번호가 같고 근거 문구가 들어 있으면 **'근거N'**, 아니면 **'무관'**

3 비교 전 공백을 모두 제거 → 줄바꿈·청크 경계 차이 무시

4 근거 문구 24개가 원문에 실제로 있는지 먼저 확인 (전부 O)

예: ['근거1', '무관', '무관', '무관']

→ 이 목록을 지표 함수에 그대로 입력

유형 배분: 단일 문서 사실형 / 복수 근거 종합형 / 정답 없는 질문(약 10~20%)으로 환각 거부 능력까지 측정

검색 지표 함수 직접 구현 — LangChain에는 내장 함수가 없음

```python
def hit_at_k(retrieved, relevant, k):
    for chunk in retrieved[:k]:
        if chunk in relevant:
            return 1
    return 0

def recall_at_k(retrieved, relevant, k):
    found = set(retrieved[:k]) & set(relevant)
    return len(found) / len(relevant)

def reciprocal_rank(retrieved, relevant, k):
    for rank, chunk in enumerate(retrieved[:k], start=1):
        if chunk in relevant:
            return 1 / rank
    return 0.0

def ndcg_at_k(retrieved, relevant, k):
    dcg, seen = 0.0, set()
    for rank, chunk in enumerate(retrieved[:k], start=1):
        if chunk in relevant and chunk not in seen:
            dcg += 1 / math.log2(rank + 1); seen.add(chunk)
    n = min(len(relevant), k)
    idcg = sum(1 / math.log2(r + 1) for r in range(1, n + 1))
    return dcg / idcg if idcg else 0.0
# + precision_at_k, context_precision_at_k, retrieval_metrics
```

강의자료 예시로 검증

K=4, 정답 {c1, c3, c5}, 검색 [c1, c7, c3, c9]

지표	결과
Hit@4	1
Precision@4	0.50
Recall@4	0.67
MRR	1.00
NDCG@4	0.70
Context Precision@4	0.83

입력은 모두 같음

retrieved 검색 결과(순위대로)
relevant 정답 모음 **k** 상위 몇 개

평가 결과 종합 — Dense 검색, K=4, 25문항

검색 평가 (정답 근거를 찾았나?)

Hit@4	Precision@4	Recall@4	MRR	NDCG@4	Context Precision
91%	25%	91%	0.76	79%	0.75

응답 평가 (맞게 답했나?) · 운영 지표

Faithfulness	Answer Relevancy	Correctness	Negative Rejection	평균 컨텍스트	평균 응답 시간
0.96	0.73	88%	100%	1,746자	4.4초

해석: 정답은 잘 찾지만(Recall 91%) 4개 중 3개는 노이즈(Precision 25%) · 정답률 88%

다음 단계: Top-K 축소·리랭커로 노이즈 감소 검토 → 단, Recall(누락) 변화와 교차 확인

사례 분석 — 정답 없는 질문 & Top-K 축소

② 정답 없는 질문 (3문항)

- 프롬프트 규칙('모르면 모른다') 적용 → 3문항 모두 **거부 성공**
- 1위 청크 유사도 평균: 사실형 **0.664** vs 정답없음 **0.493** (최대 0.540)
- 유사도 임계값 **0.55** 설정 → 정답없음 질문은 LLM 호출 없이 바로 거부
- **주의:** 정답 있는 q19(혈중알코올농도)도 임계값에 걸림 → 임계값은 별도 검증용 질문으로 결정

③ Top-K 4 → 2 축소 비교

지표	K=4	K=2
Precision	0.250	0.386
Recall	0.909	0.705
MRR	0.761	0.705
Faithfulness	0.96	0.92
Correctness	0.88	0.84
컨텍스트	1,746자	878자
응답 시간	4.4초	3.6초

노이즈·비용은 줄지만 누락(Recall)과 정답률 하락 → **K=4 유지**

핵심: 좋은 검색은 좋은 응답의 필요조건 — K·임계값은 정확도와 비용의 균형으로 결정

청킹 전략 비교 & 검색 설정 Grid Search

청킹 전략별 검색 지표 (K=4)

전략	청크 수	Hit	MRR	NDCG
고정 크기	3,400	0.955	0.663	0.707
재귀(조 경계)	4,540	0.864	0.742	0.755
최적(규정 맞춤)	**4,150**	**0.909**	**0.761**	**0.793**

- 최적 청킹이 MRR·NDCG 최고 → 정답을 더 위로 올림
- 고정 크기는 Hit은 높지만 순위가 낮음 (조문 중간 절단)
- 실험은 InMemoryVectorStore로 수행 (LLM 호출 없음)

청크 크기 × Top-K (9가지 조합)

청크	K	Hit	Recall	NDCG	컨텍스트
300	2	0.727	0.705	0.666	519자
300	4	0.818	0.773	0.701	1,041자
300	8	0.955	0.932	0.757	2,046자
500	2	0.727	0.705	0.693	860자
500	**4**	**0.909**	**0.909**	**0.793**	**1,713자**
500	8	1.000	1.000	0.822	3,323자
1000	2	0.636	0.614	0.575	1,681자
1000	4	0.864	0.818	0.680	3,434자
1000	8	0.909	0.864	0.697	6,892자

결정: 청크 500 · K=4 (정답률 91%) — K=8은 컨텍스트 2배, 청크 1000은 정확도 하락

실패 포인트(FP) 진단 & 실습 정리

진단 결과	K=4	K=2
OK	21	20
FP2 랭킹 누락	1	4
FP5 잘못된 포맷	1	1
FP6/FP7 구체성·완전성	2	0

q07 원격수업 20% 한도 → 정답 청크 미검색 (FP2)
q05 답은 맞지만 출처 표기 누락 (FP5)
q11 근거는 받았으나 '제적'을 '퇴학'으로 답변 (FP6/7)
q06 '7학점 이내'를 학기당 기준으로 확대 해석 (FP6/7)

→ K=2로 줄이면 FP2(랭킹 누락)가 1 → 4건으로 증가

실습 정리

1 **RAG = 찾아보고 답하기** LLM이 모르는 규정(134학점)도 근거와 함께 정답

2 **데이터 준비가 성능 좌우** 주석 제거·조 경계·머리말 반복 → MRR·NDCG 향상

3 **검색·응답 분리 평가** 검색 지표는 LangChain에 없어 함수로 직접 구현

4 **설정은 실험으로 결정** Golden Set으로 청크 크기·K·임계값의 균형점 도출

5 **단일 지표 의존 금지** Precision=노이즈, Recall=누락, MRR/NDCG=순서, Faithfulness=환각